ОТЧЕТ ПО ИЗ №2 АКОС — САЙГИТОВ УЛЛУБИЙ

Дисциплина: «Архитектура компьютера и операционные системы» (АКОС)

Группа: БПИ 258

Студент: Сайгитов Уллубий Русланович

ВУЗ / Год: НИУ ВШЭ, ФКН, ДПИ | 2026

Вариант: №20 («Автогонки»)

1. СВОДНАЯ МАТРИЦА ВЫПОЛНЕНИЯ ТРЕБОВАНИЙ (CHECKLIST)

Требование из ТЗ	Статус	Детали реализации в проекте
Многопоточность (pthreads)	Выполнено	Каждая машина управляется отдельным потоком `pthread_t` в `RunCar`.
Синхронизация ресурсов	Выполнено	Мьютексы клеток `pthread_mutex_t` + `pthread_cond_timedwait` по `CLOCK_MONOTONIC`.
Интерфейс CLI (getopt_long)	Выполнено	Обработка флагов: `-t` (команды), `-l` (круги), `-d` (задержка), `-m` (карта), `-o` (лог).
Наглядность и ANSI UI	Выполнено	Отрисовка карты в терминале, цветные авто, HUD и динамический Leaderboard.
Обработка сигналов (SIGINT)	Выполнено	Обработчик `signal(SIGINT, ...)` вызывает `TerminateRace` (флаг `isInterrupted`).
Загрузка конфигурации/карт	Выполнено	Чтение схемы трассы из файла, авто-построение графа и BFS-расчетов.
Логирование и статистика	Выполнено	Сохранение текстового отчета гонки с результатами через `SaveRaceLog`.
Корректное завершение	Выполнено	Сигнал `pthread_cond_broadcast` и полная синхронизация через `pthread_join`.
Прозрачность ИИ (AI Log)	Выполнено	Зафиксированы промты генерации тестов/документации и ручная адаптация кода.
Комментирование кода	Частично	Заголовочные файлы и ключевая бизнес-логика подробно прокомментированы.

2. АРХИТЕКТУРА И ОТОБРАЖЕНИЕ СУЩНОСТЕЙ НА ПОТОКИ

Поток автомобиля (Car Thread)

Каждый автомобиль выполняется в отдельном POSIX-потоке `pthread_t` (функция `RunCar`). Поток рассчитывает оптимальный ход (гибрид `TryBestMove` по BFS и `TryRandomMove`) и запрашивает атомарный переход между клетками трассы.

Главный поток-координатор (UI / Main)

Поток `main` парсит аргументы, загружает трассу, инициализирует потоки машин, организует цикл рендеринга ANSI-интерфейса с интервалом `delayMs`, отслеживает завершение и выполняет `pthread_join`.

3. СИНХРОНИЗАЦИЯ И ЗАЩИТА ОТ СОСТОЯНИЯ ГОНКИ (RACE SAFETY)

- **Мелкогранулярные мьютексы клеток:** Каждая клетка `Cell` имеет индивидуальный мьютекс `pthread_mutex_t mutex`. Переход машины в клетку выполняется атомарно через `TryTakeCell` с последующим освобождением текущей клетки `LeaveCell`, что гарантирует абсолютную защиту от наложения машин.
- **Условные переменные и точный тайминг:** Задержки потоков осуществляются через `pthread_cond_timedwait(&ctx->cond, &ctx->mutex, &ts)` с использованием монотонных часов `CLOCK_MONOTONIC` и `rand_r(&seed)`, что предотвращает «холостые» циклы и обеспечивает мгновенный отклик на остановку гонки.
- **Потокобезопасные снимки (Snapshots):** Судья (`judge.c`) и рендеринг (`race.c`) считывают состояния машин через краткосрочный захват `car->mutex` для создания локальной копии `CarSnapshot`, исключая Data Race при отрисовке.

4. ОБРАБОТКА СИГНАЛОВ, ЛОГИРОВАНИЕ И ЗАВЕРШЕНИЕ

> **Перехват SIGINT и аварийный выход**
>
> Сигнал прерывания `Ctrl+C` обрабатывается через `signal(SIGINT, ...)`, перенаправляющий обработку в `RaceSignalHandler(int sig)`, где вызывается `TerminateRace(&race)` и выставление флага `isInterrupted = true`.

> **Штатное завершение и выгрузка логов**
>
> Главный цикл отслеживает завершение гонки. Выставляется `ctx.isOver = true`, рассылается `pthread_cond_broadcast(&ctx->cond)` для разблокировки всех спящих потоков, после чего происходит сбор всех ресурсов с помощью `pthread_join`. Функция `SaveRaceLog` сохраняет итоговую статистику и Leaderboard в файл.

5. РЕАЛИЗАЦИЯ ПОЛЬЗОВАТЕЛЬСКОГО ИНТЕРФЕЙСА (CLI)

Программа предоставляет наглядный и информативный интерфейс командной строки (CLI) с подсвеченным выводом событий в реальном времени:

- **Интерактивный и CLI-режимы:** Поддерживается запуск с аргументами командной строки или в диалоговом режиме при отсутствии ключей.
- **Наглядная визуализация:** Отрисовка трассы в терминале, цветные автомобили, динамический HUD и онлайн-таблица лидеров.
- **Регулируемая задержка:** Флаг `--delay` позволяет настроить задержку для удобного наблюдения за ходом имитации.

6. ОТЧЕТ ОБ ИСПОЛЬЗОВАНИИ ИИ (PROMPTS & AI ANALYSIS)

Искусственный интеллект использовался как вспомогательный инструмент для генерации тестового окружения и документации.

> **Использованные промты:**
>
> 1. *«Напиши bash-скрипт run_tests.sh для автоматического тестирования гонки с различными картами, количеством кругов (--laps) и флагами командной строки.»*
> 2. *«Напиши технический отчет по ИЗ №2 АКОС в формате PDF со сводкой архитектуры и синхропримитивов.»*

Критический анализ и ручные правки: Сгенерированный ИИ скрипт запуска `run_tests.sh` первоначально содержал некорректные относительные пути к исполняемому файлу и не учитывал код возврата при завершении по `SIGINT`. Скрипт был вручную доработан. Также были вручную устранены неточности в оформлении документации.

7. ИНСТРУКЦИЯ ПО СБОРКЕ И ЗАПУСКУ

Выполняйте команды строго **последовательно сверху вниз**:

```
# 1. Создание папки сборки и компиляция через CMake
mkdir -p build && cd build && cmake .. && make

# 2. Переход в директорию бинарников
cd ../bin

# 3. Базовый запуск гонки
./race

# 4. Запуск с параметрами (5 кругов, кастомная карта, задержка)
./race --laps 5 --map ../data/twisty --delay 30

# 5. Кастомизация команд и сохранение лога
./race -t "RedBull:cyan:Max,Liam" -t "Ferrari:red:Charles,Lewis" --laps 3 -o my_race.log

# 6. Запуск автоматизированного набора тестов из корня
chmod +x ../run_tests.sh && ../run_tests.sh
```